

Multiagente de consulta de arquivos CSV

Desafio 4 — implementação final validada

Objetivo

Entregar um MVP que receba um ZIP com arquivos CSV e um dicionário opcional, processe os dados automaticamente, aceite perguntas em linguagem natural e retorne respostas em texto, tabela ou gráfico.

Status	Resultado
Execução	Validada offline com nove testes automatizados e onze consultas sobre a base 202401_NFs.
Framework	LangChain, com structured output Pydantic e fallback local determinístico.
Interface	Streamlit, com carga, consulta e exportação de evidências.

1. Framework utilizado

O requisito de framework é atendido com LangChain. O agente opcional é criado por create_agent, usa ChatAnthropic e solicita saída estruturada no modelo Pydantic QueryPlan. O plano contém tabela, colunas, agrupamento, agregação, ordenação, limite e tipo de saída.

Quando não há ANTHROPIC_API_KEY, o sistema usa o mesmo contrato de plano com um agente local determinístico. Isso mantém a demonstração reprodutível e evita inserir segredo no código-fonte.

Dependências principais

Biblioteca	Uso
LangChain	Orquestração do agente e structured output.
langchain-anthropic	Integração opcional com o modelo Anthropic.
Pydantic	Validação do QueryPlan antes da execução.
Pandas	Leitura e agregação determinística dos DataFrames.
Streamlit / Plotly	Interfaces e visualizações do MVP.

2. Arquitetura

A arquitetura separa interpretação de consulta e execução de dados. O modelo, quando habilitado, nunca recebe permissão para executar Python arbitrário.

Usuário	ZIP + pergunta	Resposta
Streamlit	LoaderAgent → SchemaAgent	VizAgent
QueryAgent LangChain / Local	QueryPlan Pydantic	ExecutorAgent + Pandas

Fluxo: upload → validação → schema → plano estruturado → execução controlada → texto/tabela/gráfico.

3. Agentes e responsabilidades

Agente	Responsabilidade
LoaderAgent	Valida o ZIP, impede caminhos inseguros e carrega CSVs em memória.
SchemaAgent	Inferre tipos, nulos, cardinalidade e exemplos; exibe dicionário opcional.
QueryAgent	Interpreta a pergunta com LangChain ou fallback local e produz QueryPlan.
ExecutorAgent	Aplica filtros, soma, média, contagem, distinct, agrupamento e ordenação via Pandas.
VizAgent	Escolhe texto, tabela ou gráfico e permite exportar evidências.
FallbackAgent	Mantém a aplicação funcional quando a API está ausente, indisponível ou inválida.

4. Tratamento dos dados

O carregador tenta codificações comuns em bases brasileiras, aceita separadores usuais e identifica nomes de dicionário como `data_dictionary.csv`, `dicionario.md` e `layout.txt`. A execução não altera o `DataFrame` de origem e converte números brasileiros, como 3.371.754,84, para cálculo semântico.

A contagem de notas usa `count_distinct` da CHAVE DE ACESSO quando a pergunta pede notas. Consultas de CFOP e NCM usam a tabela de itens; rankings de notas preservam chave, valor, data e emitente na resposta.

5. Perguntas e respostas verificadas

A bateria foi executada com a base 202401_NFs do material do curso: 100 notas no cabeçalho e 565 linhas de itens.

Pergunta	Resposta verificável
Qual o valor total das notas fiscais emitidas?	R\$ 3.371.754,84
Quais os 5 emitentes com maior valor total de notas?	CHEMYUNION LTDA em primeiro, com R\$ 1.292.418,75; cinco grupos retornados.
Qual UF do destinatário recebeu mais notas fiscais?	DF, com 26 notas distintas.
Qual a média de valor por nota fiscal?	R\$ 33.717,55.
Qual o CFOP mais frequente nas notas?	CFOP 6906, com 277 linhas de itens.
Qual a quantidade total por NCM?	NCM 49019900 em primeiro, com 68.684 unidades.

6. Evidência de execução

Verificação	Resultado
Testes automatizados	9 testes aprovados com unittest.
Consultas da bateria	11 perguntas executadas sobre a base real.
Média	Calculada sobre o valor do cabeçalho, não por agrupamento indevido de valores distintos.
Ranking	Ordenação numérica corrigida e limite de 5/10 aplicado conforme a pergunta.

7. Como testar

Instale as dependências com `pip install -r requirements.txt` e execute `streamlit run app.py`. Envie `sample_data/desafio4_sample.zip` para uma demonstração rápida. O roteiro completo está em `docs/GUIA_DE_TESTE.md`.

Teste automatizado

`python -m unittest discover -s tests -v` deve terminar com OK.

8. Limitações e próximos passos

O MVP cobre perguntas baseadas em colunas, agregações e agrupamentos previstos. Não cobre joins complexos, reconciliação fiscal ou regras tributárias específicas. Em uma evolução, pode-se adicionar catálogo persistente, autenticação, mais ferramentas LangChain, cache de schema e validações de domínio.

9. Entregáveis

Arquivo	Conteúdo
<code>desafio4_entrega_final.zip</code>	Pacote de código, testes, dados de exemplo, guia e documentação.
<code>relatorio_tecnico_desafio4_final.pdf</code>	Este relatório técnico.
<code>documento_estado_anterior.docx</code>	Análise do material recebido.
<code>documento_estado_atual.docx</code>	Descrição da implementação atual.
<code>comparativo_entregas.docx</code>	Comparação para compartilhamento com o integrante.